

SLIDE 1 – Introdução

1 - Segundo Frederick Brooks, desenvolvimento de software enfrenta dificuldades essenciais (para as quais não há bala de prata) e acidentais (para as quais existe uma solução melhor). Dê um exemplo de dificuldade acidental que já tenha experimentado ao desenvolver programas, mesmo que pequenos. Sugestão: elas podem estar relacionadas a ferramentas que tenha usado, como compiladores, IDEs, bancos de dados, sistemas operacionais, etc.

R: Um exemplo de dificuldade acidental que eu experimentei ao desenvolver algum programa foi quando, utilizando a linguagem de programação python, me vi perdido em diferentes ambientes virtuais (venv) em que cada um possuía uma biblioteca que eu havia instalado, porém deveria ter instalado no mesmo ambiente e isso me tomou bastante tempo e me rendeu bons erros.

2 - Diferencie requisitos funcionais de requisitos nãofuncionais. Cite um exemplo.

R: Os requisitos funcionais são as funcionalidades essenciais de um sistema, quais serviços ele deve oferecer. Exemplo: Um sistema como o github deve oferecer suporte para alocar códigos fonte de diferentes usuários.

Já os requisitos não-funcionais, para além de possuir as funcionalidades essenciais, deve ter um escopo de restrições e com que qualidade operar. Exemplo: O github, agora que aloca códigos fonte de diversos usuários, pode implementar a manutenibilidade, resolvendo bugs em um tempo específico.

3 - Explique porque testes podem ser considerados tanto uma atividade de verificação como de validação de software. Qual tipo de teste é mais adequado se o objetivo for verificação? Qual tipo de teste é mais adequado se o objetivo for validar um sistema de software?

R: Testes servem tanto para verificação quanto validação porque cobrem aspectos diferentes da qualidade do software. A verificação usa testes para garantir que o sistema foi construído conforme o planejado, checando se cada parte funciona como especificado, como testes unitários e de integração são os mais usados, pois validam detalhes técnicos.

4 - Por que testes não conseguem provar a ausência de bugs?

R: Por conta do escopo de verificação de um teste, uma vez este teria que englobar todas as possibilidades no uso de um software para que seja dado o veredito sobre a ausência de bugs. O que, a depender do tamanho do software, isto é impossível.

5 - Alguns estudos mostram que os custos com manutenção e evolução podem alcançar 80% ou mais dos custos totais alocados a um sistema de software, durante todo o seu ciclo de vida. Explique porque esse valor é tão alto.

R: Com o passar das modificações, inserções e alterações em um software, é de se imaginar que sua complexidade aumente, de maneira que apenas uma modificação é capaz de gerar uma cascata de consequências em outras dependências do software.

6 - Refactoring é normalmente definido como uma transformação de código que preserva comportamento. Qual o significado da expressão preservar comportamento? Na prática, qual restrição ela impõe a uma operação de refactoring?

R: Ao aplicar um refactoring em um código, apenas o que deve ser modificado é apenas “estético” no resultado final, como exemplo modificar nomes de variáveis. Porém, qualquer outra modificação que altere comportamento externo do software, evitando prejudicar o desempenho.

7 - Se considerarmos seu contexto histórico, por que foi natural que os primeiros processos de desenvolvimento de software tivessem características sequenciais e que fossem baseados em planejamento e documentação detalhados?

R: A natureza crítica, científica, rigorosa e imutável advinha de um período em que tais projetos se baseavam em tentar gerenciar o mais criteriosamente possível a aplicação, como em projetos de engenharia que são complexos e detalhados para garantir previsibilidade. No entanto, hoje em dia a construção de software precisa ser ágil e bem imprevisível, caracterizada por mudanças rápidas.

8 - Em 2015, descobriu-se que o software instalado em mais de 11 milhões de carros da Volkswagen detectava quando eles estavam sendo testados em um laboratório de certificação. Nessas situações, o carro emitia poluentes dentro das normas legais. Fora do laboratório, emitia-se mais poluentes, para melhorar o desempenho. Ou seja, o código provavelmente incluía uma estrutura de decisão como a seguinte (meramente ilustrativa, para fins deste exercício):

```
if "Carro sendo testado em um laboratório"  
    "Emita poluentes dentro das normas"  
else  
    "Emita poluentes fora das normas"
```

O que você faria se seu chefe pedisse para escrever um if como o acima?

R: Bom, como dependo do emprego eu tentaria explicar que isso seria anti ético, mas com a insistência escreveria o código.

9 - Dê exemplos de sistemas A (Acute, ou críticos) e B (Business, ou comerciais) com os quais já tenha interagido.

R: Já interagi com sistemas de metrô e aviários, que se configuram como sistemas A
Já interagi também com sistemas de finanças bancárias, que se configuram como sistemas B

10 - Dê exemplos de sistemas C (casuais) que você já tenha desenvolvido.

R: Já desenvolvi um sistema de finanças pessoais, para gerenciamento de débitos e créditos.